

SporTech Innovation Lab Pvt Ltd

Multi-Agent Database Management — Take-Home Assignment

Field	Details
Candidate	Jayesh Chaudhari
Time Limit	48 hours from time of issue
Language	Python 3.10+ Standard library only SQLite
Submission	ZIP file: code + 6 sample articles + run_log.txt + README.md

1. Background

You are building on top of the Sports Intelligence Compiled Knowledge Store (CKS) — a pre-indexed store of sports articles that answers fan and analyst queries fast. Once articles are ingested into the database, the data needs to stay accurate and up to date: records get misclassified, fields go missing, and source files get edited over time.

This assignment asks you to build a small Multi-Agent Framework in Python that manages the lifecycle of CKS records across four operations: populate, check, correct, and update. Each operation is handled by a dedicated agent class. An Orchestrator ties them together.

2. What You Are Building

Four agents + one Orchestrator, all in plain Python using SQLite:

Agent	One-line job
Ingestion Agent	Read article files from a folder → extract metadata → INSERT into DB
Audit Agent	Scan every DB record → flag quality problems into an audit_log table
Correction Agent	Read the audit_log → auto-fix what can be fixed → update DB records
Update Agent	Detect source files that changed on disk → refresh the DB record
Orchestrator	CLI entry point that runs agents individually or as a full pipeline

Agents communicate through the database, not by calling each other directly. The Audit Agent writes to audit_log; the Correction Agent reads from it. This keeps them independent.

3. Design Decisions — Yours to Make

The following are intentionally left open. Your README must briefly explain the choices you made and why.

Area	What we expect from you
Database schema	Design your own SQLite tables to support all four agent operations. There is no prescribed schema.
Metadata extraction	Decide what metadata to extract from each article and how. No external NLP libraries — standard library only.
Audit rules	Define at least 3 quality rules your Audit Agent will check for. Document each rule and its severity in your README.
Agent interfaces	Method signatures, return values, and internal structure are yours to decide. Each agent must be runnable independently.
Orchestrator CLI	Design the CLI commands however makes sense. It must support running agents individually and as a full pipeline.
Project structure	Organise your files however you like. Your README must explain the layout so the evaluator can navigate it cold.

4. Agent Responsibilities

Each agent has one job. How it does that job is up to you.

Ingestion Agent

Read .txt and .md article files from a folder, extract metadata from each, and insert records into the database. Skip files already in the database. Support a dry-run mode that prints what would be inserted without writing anything.

Audit Agent

Scan every record in the database and flag quality issues into an audit log. Apply your own quality rules. Assign a quality score to each article based on the issues found — an article with no issues scores highest; each issue reduces the score.

Correction Agent

Read open entries in the audit log and fix what can be fixed programmatically. Some issues will not be auto-correctable — handle those gracefully. After applying a fix, verify it worked. Update the audit log entry to reflect the outcome.

Update Agent

Detect articles whose source file has changed on disk since ingestion. Re-extract metadata from the updated file, refresh the database record, and trigger a fresh audit on that article.

Orchestrator

A CLI entry point that lets the user run any single agent or the full pipeline in sequence. It must also support a report command that prints a summary of the current database state.

5. Sample Articles

Create 6 short articles (100–250 words each) covering at least 3 different sports. Deliberately introduce quality problems into at least 3 of them — the kind your Audit Agent is designed to catch. Document each seeded problem and the expected audit finding in your README. Use the Update Agent test case to modify one article after initial ingestion and verify the refresh works.

6. Deliverables

1. Working code for all four agents and the Orchestrator.
2. 6 sample articles with at least 3 seeded quality problems.
3. `run_log.txt` — terminal output from a full pipeline run followed by the report command.
4. `README.md` — how to run the project, your schema and design decisions, your audit rules, and the seeded errors with expected findings.

7. Evaluation Criteria

Area	Weight	What We Look For
Agent correctness	35%	Each agent does its job correctly; seeded problems are caught and fixed as expected
Design thinking	25%	Schema, metadata fields, audit rules, and CLI are well-reasoned and explained in README
Code quality	25%	Clean separation of concerns between agents; readable, commented code
README & run log	15%	Evaluator can run the project cold; design decisions and seeded errors are clearly documented

8. Rules

- Python 3.10+ only.
- Standard library only — `sqlite3`, `json`, `hashlib`, `re`, `pathlib`, `argparse`, `logging`, `datetime`. No pip installs.
- No AI or LLM API calls. All logic must be rule-based.
- Each agent must be importable and runnable without the Orchestrator.
- Code must run on a fresh Python 3.10 install with zero configuration beyond pointing `--source` at the `sample_articles` folder.

Where to start

Build `shared/db.py` first (schema creation). Then build `IngestionAgent` and test it against 2 sample articles before touching anything else. The `Audit Agent` has the most logic — budget extra time there. Build the `Orchestrator` last.

Submit as a ZIP to your point of contact within 48 hours of receiving this. Questions? Reach out within the first 12 hours.